Proyecto 1: Sistema de administración académica y navegación de contenidos educativos.

Oct 13, 2026.

Patrick Zúñiga Arroyo - 2025104944

1. Introducción

Este proyecto es un sistema de consola en C++ que administra estudiantes y cursos, y permite navegar el contenido educativo de cada curso. Cada curso es un árbol de cursos, unidades, temas, recursos y actividades. Cada estudiante tiene un historial de páginas visitadas, recursos completados y actividades inscritas, con opción de retroceder y deshacer. También hay una cola de solicitudes con prioridad, un índice de estudiantes por carné (árbol balanceado), once reportes y persistencia en archivos de texto.

Alcance. Todo lo que pide el enunciado está implementado y probado. También se hicieron tres extras: el balanceo del índice (árbol AVL), la ruta entre dos nodos del árbol genérico y las estadísticas de uso de recursos (sección 10).

Compilar y ejecutar (Windows, MinGW g++ 6.3). El compilador usa C++14:

```
g++ -std=c++14 -Wall -Wextra -g main.cpp Utils.cpp Estudiante.cpp
HistorialAcademico.cpp GestorEstudiantes.cpp ArbolGenerico.cpp GestorCursos.cpp
GestorNavegacion.cpp GestorSolicitudes.cpp Reportes.cpp Persistencia.cpp Menus.cpp
MenuNavegacion.cpp MenuSolicitudes.cpp MenuIndiceReportes.cpp -o proyecto.exe
```

También se puede usar `compilar.bat` y luego `.\proyecto.exe`.

2. Arquitectura modular

Cada módulo tiene un .h (qué ofrece) y un .cpp (cómo lo hace). Las plantillas (ABB) viven solo en el .h.

Archivo	Funcionalidad
Utils	Lectura y validación de entradas, manejo de texto (namespace Utils).
Estudiante	Datos del estudiante (carné, nombre, correo) y su historial.
HistorialAcademico	Pila de páginas, pila de acciones, completados e inscripciones.
ABB.h	Árbol binario de búsqueda genérico y balanceado (AVL), como plantilla.
GestorEstudiantes	Lista de estudiantes y índice ABB por carné.

Archivo	Funcionalidad
ArbolGenerico	Árbol de contenido de un curso (NodoCurso con lista de hijos), con numeración y ruta entre nodos.
GestorCursos	Lista de cursos y búsqueda de nodos por id.
GestorNavegacion	Visitar, retroceder, completar, inscribirse y deshacer.
GestorSolicitudes	Tres colas de solicitudes, modo prioridad o llegada.
Reportes	Los once reportes, escritos a pantalla o a archivo.
Persistencia	Guardar y cargar en cuatro archivos de texto.
Menus, MenuNavegacion, MenuSolicitudes, MenuIndiceReportes	Interfaz de consola de cada área.
main	Arranque, menú principal, guardar al salir.

La regla es que los módulos de datos no imprimen ni leen del teclado; eso solo lo hacen los menús y los reportes.

3. Estructuras de datos por módulo y justificación

Lista (`std::list<Estudiante>`). Guarda a los estudiantes. Se eligió lista porque insertar y borrar no mueve los demás elementos, y las direcciones de los estudiantes no cambian, lo que permite que el índice los apunte sin riesgo.

Árbol binario de búsqueda balanceado (ABB<long long, Estudiante*>). Índice por carné. Cada nodo guarda su altura y, después de insertar o eliminar, se compara la altura de sus dos lados; si la diferencia es mayor que 1 se hace una rotación (simple o doble) que lo reacomoda. Así el árbol nunca se vuelve una lista, aunque los carnés se ingresen en orden, y buscar un carné cuesta O(log n) siempre, contra O(n) de recorrer la lista. Se implementó con plantilla, inserción, búsqueda, eliminación (tres casos), mínimo, máximo, altura y los recorridos inorden, preorden y postorden. Al eliminar un estudiante se borra primero del índice y luego de la lista.

Pila (`std::stack`). El historial de cada estudiante usa dos: páginas visitadas (retroceder saca la última) y acciones realizadas (deshacer saca la última). Ambas son LIFO (“Last In, First Out”) por naturaleza: lo último que se hizo es lo primero que se revierte.

Cola (`std::queue`). Las solicitudes se atienden en orden de llegada (FIFO, “First In, First Out”). Hay tres colas, una por nivel de prioridad: alta (reporte de error), media (habilitación de recurso) y baja (revisión de tarea y consulta académica). En modo prioridad se atiende la primera cola con elementos; en modo llegada se atiende la de menor número de solicitud entre los frentes.

Árbol genérico (`NodoCurso`, hijos en `std::list<NodoCurso*>`). Un curso tiene unidades, temas, recursos y actividades con cantidad de hijos variable, por lo que un árbol con lista de hijos es lo natural. Cada nodo guarda su padre, un id único y un contador de visitas. Se calculan nivel, altura, hojas y ruta desde la raíz.

4. Integración entre módulos

Las estructuras no trabajan aisladas. Un solo flujo las usa casi todas:

1. **Registro.** `GestorEstudiantes` guarda el estudiante en la lista y su puntero en el ABB.
2. **Contenido.** `GestorCursos` mantiene la lista de cursos; cada curso es un `ArbolGenerico`.
3. **Navegación.** `GestorNavegacion` recibe el carné (se busca en el ABB) y el nodo (se busca en el árbol). Al visitar apila la página en el historial, sube el contador de visitas del nodo y apila la acción. Deshacer saca esa acción y la revierte.
4. **Solicitudes.** `GestorSolicitudes` valida que existan el estudiante (ABB) y el nodo (árbol) antes de encolar.
5. **Reportes y persistencia.** Recorren todas las estructuras y escriben a pantalla o archivo.

Ejemplo concreto: el estudiante 2024001 visita "PDF de teoría" (pila de páginas + contador del nodo), lo marca completado (lista + pila de acciones), reporta un error en ese recurso (cola de prioridad alta) y deshace el completado (sale de la pila de acciones). Todo en un mismo escenario con datos coherentes entre estructuras.

```
Menus --> GestorNavegacion --> GestorEstudiantes (lista + ABB) --> Estudiante -->
HistorialAcademico (pilas)
   |             |
   |             +-----------> GestorCursos --> ArbolGenerico --> NodoCurso
   +--> GestorSolicitudes (3 colas) --usa--> GestorEstudiantes y GestorCursos
   +--> Reportes / Persistencia --leen--> todos los gestores
```

Si se elimina un nodo del árbol, el historial y las solicitudes que lo mencionaban siguen existiendo y los reportes muestran "[nodo eliminado]" en vez de fallar.

5. Análisis de complejidad

n = estudiantes, h = altura del ABB, m = nodos del curso, k = hijos de un nodo, p = elementos de una pila o cola. Como el ABB se balancea solo, h es siempre cercana a log n, incluso con carnés ingresados en orden.

Operación	Estructura	Promedio	Peor caso
Buscar por carné	ABB	O(log n)	O(log n)
Insertar / eliminar en el índice	ABB	O(log n)	O(log n)
Buscar por nombre o correo	Lista	O(n)	O(n)

Operación	Estructura	Promedio	Peor caso
Registrar estudiante (revisa carné y correo repetidos)	Lista + ABB	O(n)	O(n)
Recorrido inorden / pre / post	ABB	O(n)	O(n)
Mínimo / máximo	ABB	O(log n)	O(log n)
Buscar nodo por id	Árbol genérico	O(m)	O(m)
Agregar hijo (sin contar la búsqueda del padre)	Lista de hijos	O(1)	O(1)
Eliminar hoja	Árbol genérico	O(k)	O(k)
Altura / contar nodos / hojas	Árbol genérico	O(m)	O(m)
Ruta de la raíz a un nodo	Árbol genérico	O(nivel)	O(m)
Ruta entre dos nodos	Árbol genérico	O(nivel)	O(m)
Estadísticas de uso (más visitados)	Árbol genérico + lista	O(m log m + n)	O(m log m + n)
Visitar página, retroceder, deshacer	Pilas	O(1)	O(1)
Insertar solicitud	Cola	O(1)	O(1)
Atender siguiente	3 colas	O(1)	O(1)
Ver todas las pendientes	Copias de colas	O(p)	O(p)
Guardar / cargar archivos	Todas	O(total)	O(total)

6. Persistencia

Los datos se guardan en cuatro archivos de texto con `|` como separador, en la carpeta del programa:

Archivo	Formato de cada línea
datos_estudiantes.txt	`carne|nombre|correo`
datos_cursos.txt	`id|idPadre|tipo|nombre|visitas`

Archivo	Formato de cada línea
datos_historiales.txt	`P|carne|id` (página), `A|carne|tipo|id` (acción), `C|carne|id` (completado), `I|carne|id` (inscripción)
datos_solicitudes.txt	`config|modo|proximo|atendidas`, luego `numero|carne|idNodo|tipo|descripcion`

"Carga todo o nada": Primero se lee todo hacia gestores temporales. Solo si no hubo ningún error (línea mal formada, carné repetido, nodo inexistente) se reemplazan los datos reales. Así un archivo dañado nunca deja el sistema a medias.

Detalles: Los textos con `|` se guardan cambiando ese carácter por `/`. Se ignora el `\r` de los archivos con saltos de línea de Windows. Las pilas se guardan de la base a la cima para recuperar el mismo orden. El menú principal ofrece guardar (7), cargar (8) y al salir pregunta si guardar.

7. Validaciones, memoria y buenas prácticas

Validaciones de entrada: Los números se leen como texto y se convierten, así que letras o vacío no interrumpen el programa. El correo debe terminar en `@estudiantec.cr`. El carné y el correo no se pueden repetir. Una solicitud exige estudiante y nodo existentes y descripción no vacía. Eliminar un nodo solo se permite si es hoja, y las acciones peligrosas piden confirmación.

Memoria: Los nodos del árbol se crean con `new` y se liberan en postorden (primero los hijos). Los destructores de `ArbolGenerico` y `ABB` liberan todo. Las clases con punteros tienen el constructor de copia borrado para evitar copias que liberen dos veces.

Buenas prácticas:

- Un módulo por responsabilidad, con .h y .cpp separados.
- `const` en los métodos que solo consultan.
- `enum class` para los tipos de nodo y de solicitud.
- Sin `std::to_string` ni nada fuera de C++14, para compilar con MinGW 6.3.
- Comentarios cortos que explican el porqué.
- Los módulos de datos no imprimen: devuelven valores o escriben a un `std::ostream`.

8. Dificultades encontradas

- **Dos dueños para un estudiante.** La lista y el ABB apuntan al mismo objeto. Hubo que fijar un orden claro: la lista es dueña y el ABB es índice, por eso se elimina primero del índice. Cambiar el carné obliga a reindexar.
- **Numeración del árbol.** El id global único no sirve para que el usuario se ubique. Se agregó una numeración por posición (1, 1.1, 1.1.1) calculada al mostrar, y el id interno se conserva para historial, solicitudes y archivos.
- **Cargar sin dañar datos.** Un archivo a medias dejaba el sistema inconsistente. Se resolvió cargando primero a gestores temporales y reemplazando solo si todo salió bien.

- **Compilador antiguo.** MinGW 6.3 no tiene `std::to_string` ni C++17; se reemplazó por `ostringstream` y se mantuvo todo en C++14.
- **Const y recorridos.** Para mostrar mínimo y máximo en reportes se hicieron versiones `const` en el ABB en vez de quitar el `const`.
- **Caracteres especiales.** El separador `|` dentro de un nombre rompía la carga; se reemplaza por `/`. Los saltos de línea de Windows (`\r`) también se limpian.
- **Tildes en consola.** Se activó con `SetConsoleOutputCP` para ver bien los acentos.
- **Balanceo del ABB.** Rotar bien exige actualizar las alturas de abajo hacia arriba y rebalancear también al eliminar, no solo al insertar. Se resolvió con una función balancear que se llama al volver de cada llamada recursiva, y se probó con las cuatro rotaciones y un método que verifica el balance de todo el árbol.

9. Limitaciones conocidas y mejoras futuras

- Las estadísticas de uso cuentan las visitas totales de cada nodo; no guardan la fecha ni qué estudiante hizo cada visita. Mejora: registrar un historial de visitas con fecha.
- Buscar por nombre o correo recorre toda la lista. Mejora: índices adicionales.
- Solo se eliminan nodos hoja. Mejora: eliminar un subárbol completo con confirmación.
- Los archivos de texto no están cifrados ni tienen control de versiones. Mejora: formato con versión o base de datos.
- La interfaz es solo de consola. Mejora: interfaz gráfica.
- El historial y las solicitudes de un nodo eliminado se conservan pero apuntan a un nodo que ya no existe (se muestra como "[nodo eliminado]").

10. Facilidades extra implementadas

- Persistencia completa en cuatro archivos con carga “todo o nada”.
- Exportar los reportes a reportes.txt (opción 12 del menú de reportes).
- Numeración jerárquica del árbol (1, 1.1, 1.1.1) para elegir nodos más sencillo.
- Modo de atención de solicitudes por prioridad o por llegada, intercambiable en ejecución.
- Curso de ejemplo cargable con un clic para demostraciones.
- Dibujo del árbol ABB de lado, para ver su forma.
- Pruebas automáticas por módulo (ver sección 12).
- Balanceo AVL del índice de estudiantes: el ABB se reacomoda solo con rotaciones al insertar y eliminar.
- Ruta entre dos nodos del árbol genérico (menú 2, opción 11): sube hasta el ancestro común y baja hasta el destino.
- Estadísticas de uso de recursos (reporte 11): contenidos más visitados y actividad de cada estudiante.

11. Manual de usuario breve

Al ejecutar `proyecto.exe` aparece el menú principal. Se elige con el número y ENTER; 0 siempre vuelve o cancela.

Opción	Para qué sirve
1. Estudiantes	Registrar, cambiar nombre, correo o carné, eliminar, buscar y listar.
2. Cursos	Crear curso, agregar o eliminar nodos, ver el árbol, ver todos los cursos, buscar nodo, información de un nodo, altura y cantidad de nodos, mostrar nodos hoja, curso de ejemplo y ruta entre dos nodos.
3. Navegación	Visitar páginas, ver historial, retroceder, completar recurso, inscribirse en actividad, deshacer, ver progreso.
4. Solicitudes	Crear solicitud, atender la siguiente, ver el frente, ver todas, contar, cambiar modo de atención.
5. Índice ABB	Buscar por carné, recorridos, menor y mayor, altura y forma del árbol balanceado.
6. Reportes	Los 11 reportes (el 11 son las estadísticas de uso) y la opción 12 para exportarlos a reportes.txt.
7 / 8	Guardar / cargar datos.
9. Salir	Salir (pregunta si guardar).

Flujo recomendado para probar:

1. Menú 1: registrar dos o tres estudiantes (el correo termina en `@estudiantec.cr`).
2. Menú 2, opción 10: cargar el curso de ejemplo y ver el árbol con la opción 4.
3. Menú 2, opción 11: probar la ruta entre dos nodos (por ejemplo de 1.1.1 a 2.1).
4. Menú 3: visitar un nodo escribiendo su número (por ejemplo `1.1.2`), completar un recurso y probar retroceder y deshacer.
5. Menú 4: crear solicitudes de distinto tipo y atenderlas en modo prioridad y en modo llegada.
6. Menú 6: ver los reportes (el 11 son las estadísticas de uso) y exportarlos con la opción 12.
7. Opción 7 y salir; al volver a abrir, la opción 8 recupera todo.

Al elegir un nodo se escribe su número completo (`1.1.2`), `c` para el curso raíz y `0` para cancelar.

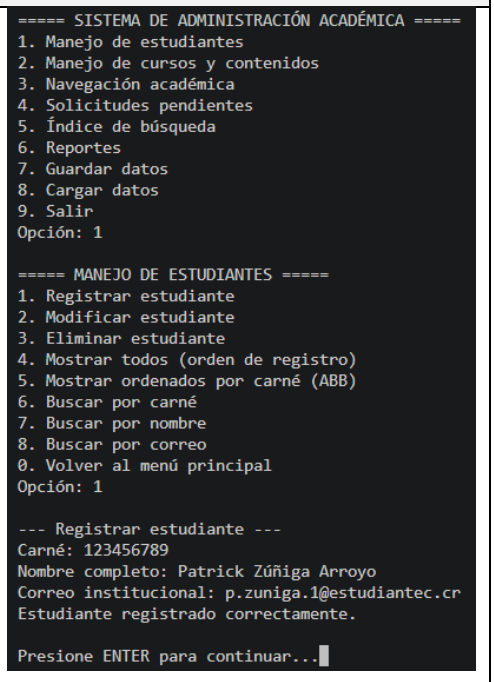
12. Casos de prueba

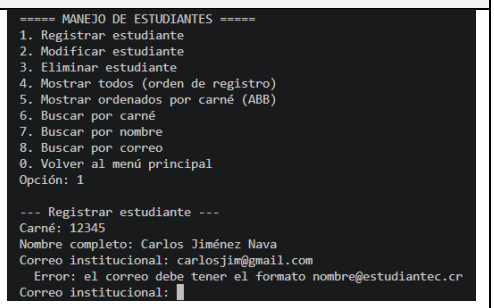
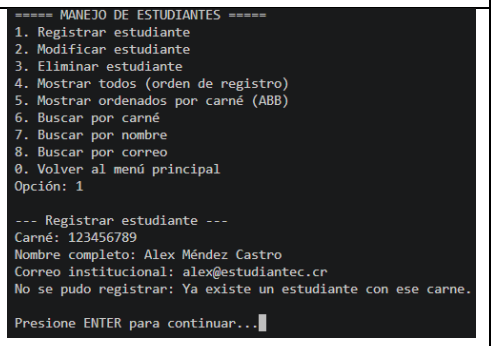
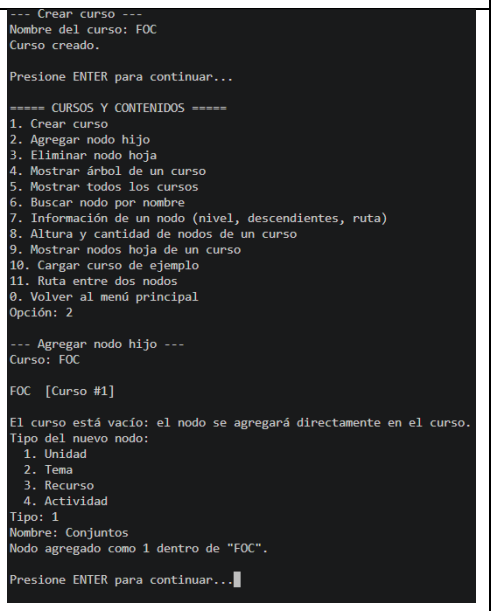
Pruebas automáticas. Cada módulo tiene su programa de prueba con su propio main y un .bat para correrlo (por ejemplo probar_abb.bat). Total: 426 comprobaciones, todas pasan.

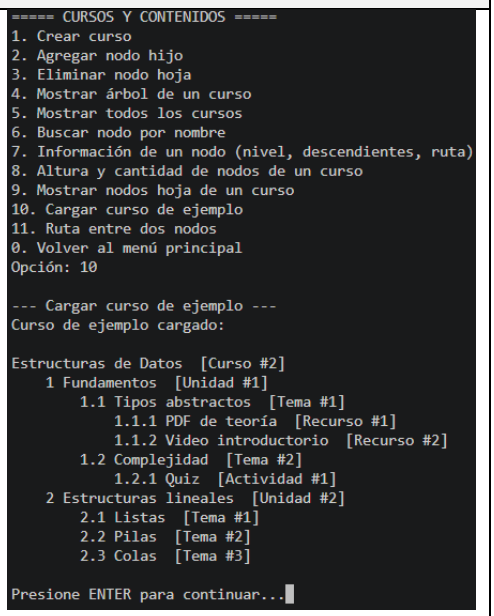
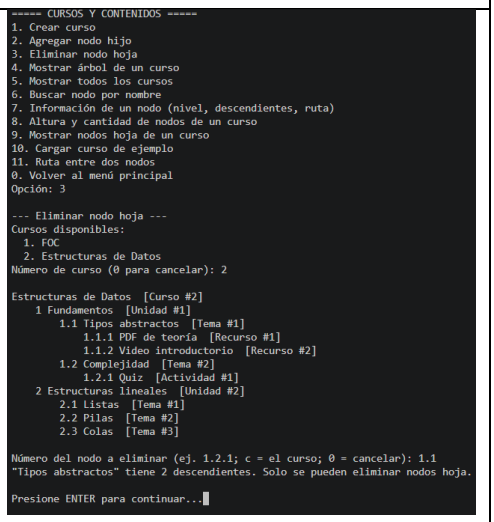
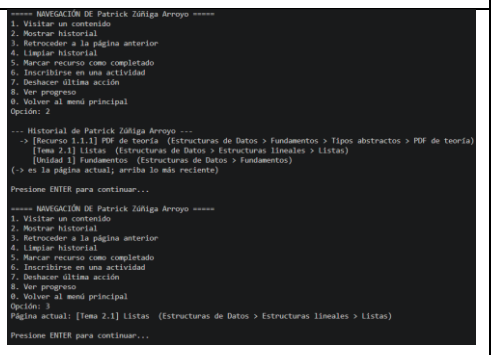
Programa	Qué verifica	Comprobaciones
pruebaABB	Insertar, buscar, eliminar (3 casos), recorridos, mínimo,	65

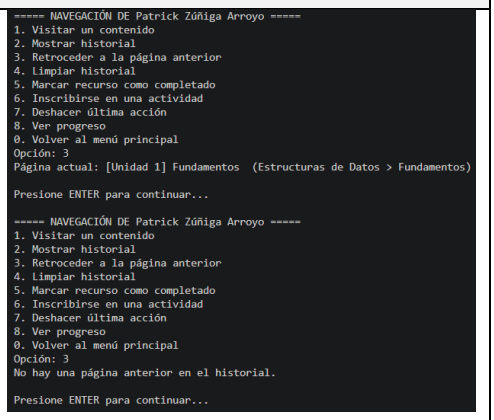
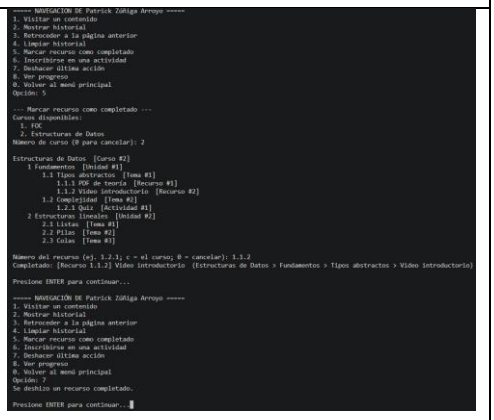
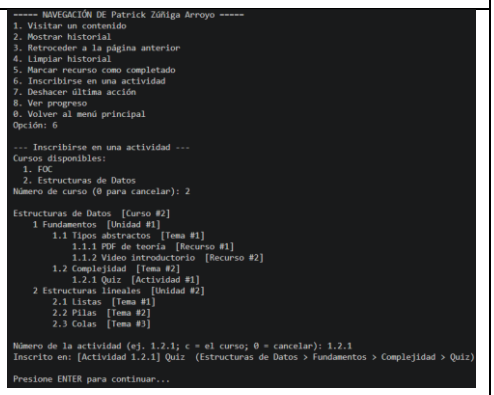
Programa	Qué verifica	Comprobaciones
	máximo, altura y balanceo AVL (4 rotaciones)	
pruebaEstudiantes	Registro, duplicados, cambios, eliminación, índice	82
pruebaCursos	Árbol genérico, numeración, ruta, ruta entre nodos, hojas, altura, eliminar	154
pruebaNavegacion	Visitar, retroceder, completar, inscribir, deshacer	41
pruebaSolicitudes	Colas, prioridad, llegada, validaciones	26
pruebaReportes	Los 11 reportes y la exportación	22
pruebaPersistencia	Guardar, cargar, archivos dañados, caracteres especiales	36

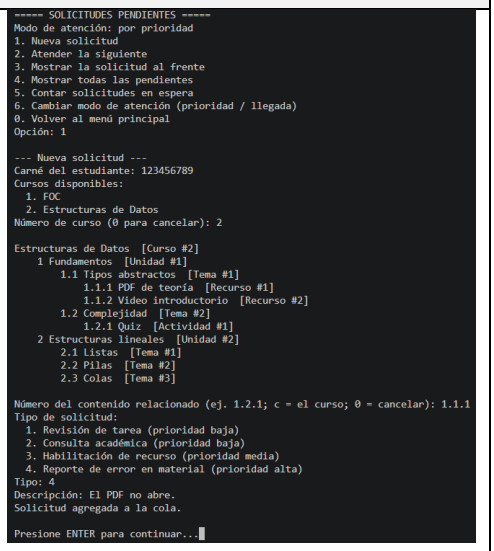
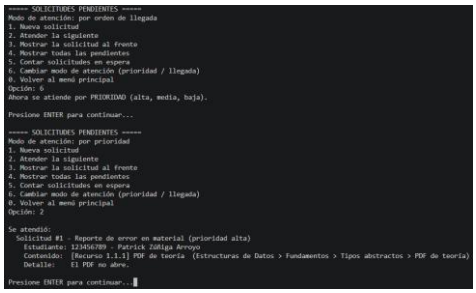
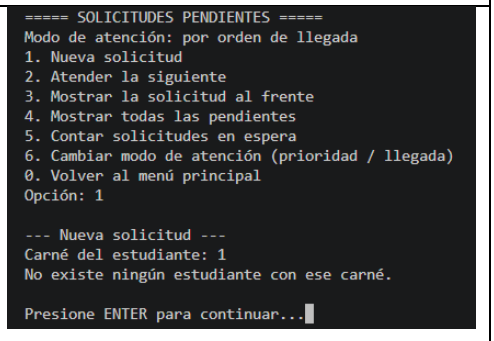
Caso manual. La columna Resultado contiene capturas de pantalla con los resultados del comando.

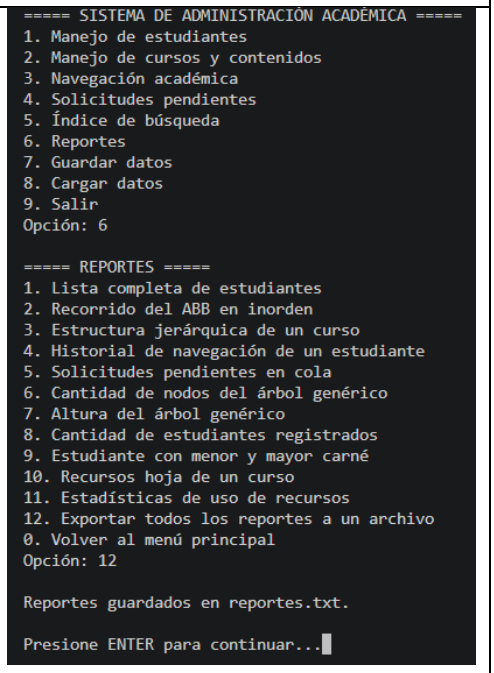
Caso	Pasos	Resultado esperado	Resultado
CP-01	Registrar estudiante con correo válido	Aparece en la lista y en el índice	

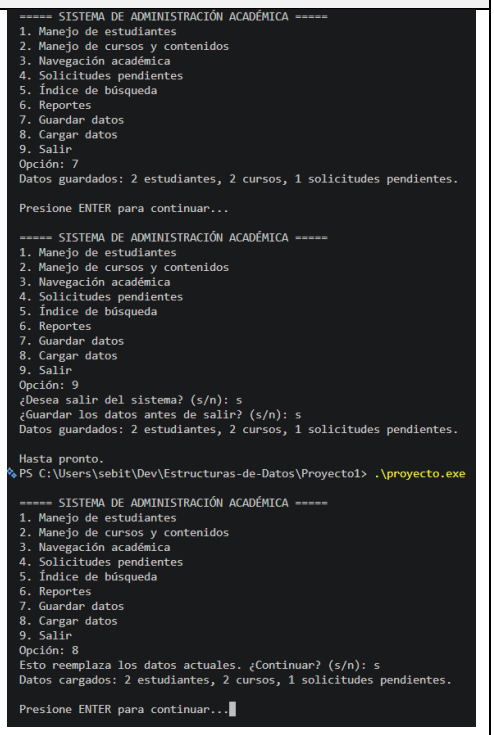
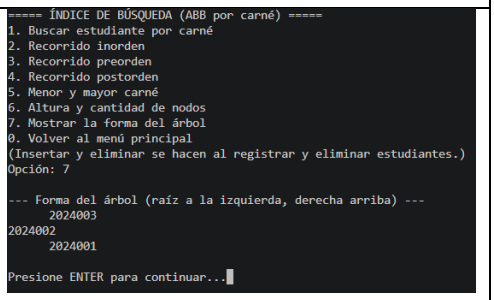
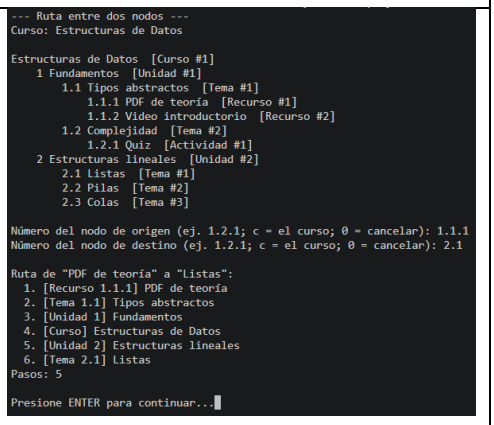
Caso	Pasos	Resultado esperado	Resultado
CP-02	Registrar con correo sin `@estudiantec.cr`	Se rechaza con mensaje	
CP-03	Registrar un carné repetido	Se rechaza con mensaje	
CP-04	Crear curso y agregar la primera unidad	Se agrega directo al curso	

Caso	Pasos	Resultado esperado	Resultado
CP-05	Cargar el curso de ejemplo y mostrar el árbol	Numeración 1, 1.1, 1.1.1	
CP-06	Eliminar un nodo que tiene hijos	Se rechaza; solo se eliminan hojas	
CP-07	Visitar tres páginas y retroceder	Vuelve a la página anterior	

Caso	Pasos	Resultado esperado	Resultado
CP-08	Retroceder con una sola página	Mensaje de que no hay a dónde volver	
CP-09	Completar un recurso y deshacer	Deja de estar completado	
CP-10	Inscribirse en una actividad	Queda inscrito	

Caso	Pasos	Resultado esperado	Resultado
CP-11	Crear solicitudes de error y atender en modo prioridad.	Sale primero el reporte de error y luego un resumen de la solicitud	 
CP-12	Cambiar a modo llegada y atender	Sale la de menor número	
CP-13	Solicitud con estudiante inexistente	Se rechaza	

Caso	Pasos	Resultado esperado	Resultado
CP-14	Buscar por carné y mostrar recorridos del índice	Inorden sale ordenado por carné	
CP-15	Exportar reportes	Se crea reportes.txt con los 11 reportes	

Caso	Pasos	Resultado esperado	Resultado
CP-16	Guardar, cerrar, abrir y cargar	Los datos vuelven iguales	
CP-17	Alterar un archivo `datos_*.txt` y cargar	Se rechaza y los datos actuales quedan intactos	
CP-18	Registrar tres estudiantes con carnés crecientes (2024001, 2024002, 2024003) y ver la forma del árbol (menú 5, opción 7)	Queda balanceado: raíz 2024002 y altura 2 (sin balanceo sería una línea de altura 3)	
CP-19	Con el curso de ejemplo, menú 2, opción 11: ruta de 1.1.1 a 2.1	Sube hasta el curso y baja: 6 nodos y 5 pasos	

Caso	Pasos	Resultado esperado	Resultado
CP-20	Visitar dos veces el mismo recurso y abrir el reporte 11	El recurso aparece primero con 2 visitas y cada estudiante con su actividad	